

Corrección de Incidencias Detectadas

Actividad N°: 33 **Fecha:** 15/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Cerrar formalmente el registro de incidencias detectadas durante todo el ciclo de desarrollo (Semanas 3 a 7), verificando que todas estén corregidas antes de pasar a la validación con usuarios de la empresa, corregir la incidencia menor registrada el día anterior, y realizar una revisión de seguridad extendida sobre archivos del proyecto que no fueron modificados durante las correcciones previas.

2. Registro consolidado de incidencias del proyecto

#	Incidencia	Detectada en	Severidad	Estado
1	<code>bulk-canje</code> autorizaba tanto a Jefe como a Staff a nivel de API, contradiciendo el diseño de interfaz (solo Jefe)	Revisión de diseño, Semana 3 (Día 15)	Media (control de acceso)	✓ Corregida en Semana 4 (Día 19)
2	Fallback hardcodeado de localidades de un evento anterior en <code>getTicketsForStaff</code> , activable si el punto de trabajo no coincidía con ningún Punto de Venta real	Desarrollo, Semana 5 (Día 21)	Alta (datos incorrectos silenciosos)	✓ Corregida en Semana 5 (Día 21)
3	Condición de búsqueda muerta ' <code>Número de Cédula: '</code> en 3 controladores, nunca coincidía con el schema real	Desarrollo, Semana 5 (Día 22)	Baja (deuda técnica, sin impacto funcional)	✓ Corregida en Semana 5 (Día 22)
4	Doble mecanismo de creación de usuario administrador (<code>database.js</code> creaba <code>admin@shakira.com</code> automáticamente, compitiendo con <code>setup.js</code>)	Validación administrativa, Semana 6 (Día 30)	Alta (credenciales documentadas podían no funcionar)	✓ Corregida en Semana 6 (Día 30)
5	Nombre de servicio ' <code>FTT-tickets</code> ' en metadatos del logger (Winston), remanente del proyecto anterior	Pruebas funcionales, Semana 7 (Día 32)	Muy baja (cosmética, solo visible en logs internos)	✓ Corregida hoy (Día 33)

3. Corrección aplicada hoy: nombre de servicio en el logger

Archivo modificado: `backend/src/config/logger.js`

```
- defaultMeta: { service: 'FTT-tickets' },  
+ defaultMeta: { service: 'canje-fft' },
```

Este cambio no afecta el comportamiento del sistema; únicamente corrige la etiqueta `service` que Winston adjunta a cada entrada de `logs/combined.log` y `logs/error.log`, alineándola con la identidad real del proyecto (Canje FTT / FeelTheTickets).

4. Verificación de cierre de todas las incidencias

Se realizó una revisión cruzada de los 5 archivos modificados durante todo el proyecto para confirmar que ninguna corrección quedó a medias:

Archivo	Cambios acumulados	Verificado
<code>backend/src/routes/tickets.js</code>	<code>authorize('jefe')</code> en <code>bulk-canje</code>	✓
<code>backend/src/controllers/puntoVentaController.js</code>	Fallback eliminado + limpieza de campo cédula (2 ocurrencias)	✓
<code>backend/src/controllers/ticketController.js</code>	Limpieza de campo cédula (1 ocurrencia)	✓
<code>backend/src/config/database.js</code>	Eliminación de <code>createDefaultAdmin()</code> y su importación de <code>User</code>	✓
<code>backend/src/config/logger.js</code>	Nombre de servicio corregido	✓

5. Casos de regresión manual para cada incidencia corregida

Con el registro de incidencias cerrado, se documentaron los casos de prueba de regresión que permiten confirmar, en cualquier momento futuro, que ninguna de las 5 incidencias vuelve a aparecer. Estos casos pueden ejecutarse manualmente o servir de base para una futura suite automatizada.

Incidencia 1 — Control de acceso en `bulk-canje`

ID	Caso	Resultado esperado
REG-01	POST <code>/api/tickets/bulk-canje</code> con token de rol <code>staff</code>	403 Forbidden
REG-02	POST <code>/api/tickets/bulk-canje</code> con token de rol <code>jefe</code> y body válido	200 OK (o 207 si algún ticket ya estaba canjeado)
REG-03	POST <code>/api/tickets/bulk-canje</code> sin token	401 Unauthorized

Incidencia 2 — Fallback hardcodeado de localidades

ID	Caso	Resultado esperado
REG-04	GET <code>/api/tickets</code> con token de Staff cuyo <code>puntoTrabajo</code> no existe en la colección <code>PuntosVenta</code>	Lista vacía [], sin devolver localidades de un evento anterior
REG-05	GET <code>/api/tickets</code> con token de Staff cuyo <code>puntoTrabajo</code> sí existe en la colección	Lista filtrada por las localidades reales de ese Punto de Venta

Incidencia 3 — Campo de búsqueda de cédula muerto

ID	Caso	Resultado esperado
REG-06	GET <code>/api/tickets?busqueda=0123456789</code> (número de cédula real)	Devuelve resultados usando el campo correcto del schema, no la cadena muerta ' <code>Número de Cédula:</code> '
REG-07	GET <code>/api/tickets?busqueda=0123456789</code> con cédula que no existe	Lista vacía [], sin error 500

Incidencia 4 — Doble mecanismo de creación de administrador

ID	Caso	Resultado esperado
REG-08	Arrancar el servidor sin haber ejecutado <code>setup.js</code> previamente	No se crea ningún usuario automáticamente al arrancar
REG-09	Ejecutar <code>node src/scripts/setup.js archivo.csv</code> sobre una base vacía	Se crea exactamente un usuario con <code>usuario: 'sistema'</code> y rol <code>'jefe'</code>
REG-10	Ejecutar <code>node src/scripts/setup.js</code> una segunda vez sobre la misma base	El script no crea un segundo usuario <code>sistema</code> (manejo de duplicados por índice único o upsert)

Incidencia 5 — Nombre de servicio del logger

ID	Caso	Resultado esperado
REG-11	Iniciar el servidor y revisar <code>logs/combined.log</code>	Cada entrada contiene <code>"service": "canje-fft"</code> , no <code>"service": "shakira-tickets"</code>

6. Revisión de seguridad extendida — Archivos no modificados durante el proyecto

Tras cerrar el registro formal de incidencias, se realizó una revisión de los archivos del backend que **no fueron modificados** durante las semanas anteriores, buscando patrones similares a los ya corregidos: valores hardcodeados, referencias a proyectos anteriores, campos muertos o configuraciones inseguras.

6.1 `backend/src/middleware/auth.js`

Revisado en busca de:

- Roles hardcodeados distintos a los definidos en el modelo `User` → No encontrados. La función `authorize(...roles)` acepta roles dinámicos como parámetro.
- Comparación de roles con cadenas que no coinciden con el enum del schema → No encontrado; los roles usados en las llamadas a `authorize()` en las rutas ('jefe', 'staff') coinciden con los definidos en el schema.

Resultado: sin hallazgos.

6.2 `backend/src/middleware/auditLogger.js`

Revisado en busca de:

- Tipos de auditoría hardcodeados que podrían no coincidir con los usados en los controladores → Se detecta que el `switch` del middleware cubre solo un subconjunto de tipos (`login`, `logout`, `change-password`), mientras que los controladores crean sus propios registros de auditoría para `canje`, `reimprimir`, etc. No es un error funcional (los registros de los controladores sí llegan a la base), pero implica que las rutas de tickets tienen auditoría duplicada (middleware + controlador). Se registra como observación; no se corrige hoy por estar fuera del alcance de esta sesión.

Resultado: una observación de arquitectura, sin impacto funcional inmediato.

6.3 `backend/src/scripts/setup.js`

Revisado en busca de:

- Nombres de evento hardcodeados del proyecto anterior → Se encontró que el nombre del Punto de Venta y las localidades se leen desde el CSV provisto como argumento, no desde valores hardcodeados. Correcto.
- Contraseñas hardcodeadas → La contraseña inicial está establecida directamente en el script como `password: 'sistema-inicial'`, sin leer ninguna variable de entorno. Esto es aceptable para el contexto (es un valor conocido y documentado que el administrador debe cambiar en el primer acceso), y no representa un riesgo equivalente al del fallback de localidades corregido en la Semana 5, cuyo problema era devolver datos silenciosamente incorrectos.

Resultado: sin hallazgos críticos.

6.4 `backend/src/models/` (todos los modelos)

Revisado en busca de:

- Nombres de colecciones hardcodeados con referencias al proyecto anterior → No encontrados. Los nombres de colección son genéricos (`Tickets`, `Users`, `PuntosVenta`, `AuditLogs`).
- Valores por defecto (`default:`) con datos de eventos anteriores → No encontrados.

Resultado: sin hallazgos.

6.5 `backend/src/controllers/authController.js` — normalización del campo `usuario`

Revisado en busca de inconsistencias entre el schema y el comportamiento del controlador:

- El modelo `User.js` define el campo `usuario` con `lowercase: true`, de modo que todo valor se persiste siempre en minúsculas.
- Sin embargo, `authController.js` realiza la búsqueda con `User.findOne({ usuario })` usando el valor **tal cual llega del formulario**, sin aplicar `.trim().toLowerCase()` antes de la consulta.
- **Impacto:** si el campo de usuario llega con alguna letra en mayúscula (por autocompletado del navegador o por error de tipeo), la búsqueda no encuentra ningún documento y el controlador responde "Credenciales inválidas" — el mismo mensaje que devuelve para una contraseña realmente incorrecta. Esto hace indistinguibles los dos casos para el usuario, y puede llevar a que operadores con credenciales válidas no puedan iniciar sesión.

Severidad: Media (puede impedir el acceso en producción sin dejar pista clara de la causa). **Recomendación:** agregar `.trim().toLowerCase()` al valor de `usuario` antes del `findOne`, replicando la normalización que el schema aplica al guardar.

6.6 `frontend/src/services/api.js` — interceptor de respuestas Axios

Revisado el interceptor de respuestas de Axios que maneja errores globales:

- El interceptor trata **cualquier** respuesta HTTP 401 como señal de "sesión expirada": limpia `localStorage` y fuerza `window.location.href = '/login'` (recarga completa del navegador).
- Esta lógica no distingue entre un 401 por token expirado en una ruta protegida (caso para el que fue diseñada) y el 401 que el propio endpoint `POST /auth/login` devuelve cuando la contraseña es incorrecta.
- **Impacto:** en el segundo caso, la recarga forzada de página ocurre antes de que el componente `Login.jsx` pueda ejecutar su bloque `catch` y renderizar el mensaje de error. El resultado visible es que la pantalla parpadea o queda en blanco, sin mostrar ningún mensaje de error al usuario.

Severidad: Media-Alta en producción (el error de credenciales queda completamente enmascarado, el operador no sabe qué falló). **Recomendación:** agregar una excepción explícita en el interceptor para que la limpieza de sesión y la redirección forzada no se activen cuando el 401 proviene de `/auth/login`, dejando que el formulario maneje ese caso de forma normal.

7. Revisión de valores hardcodedos en el frontend

Se realizó una búsqueda de cadenas hardcodedas en el código del frontend que pudieran contener referencias a otro proyecto u otros datos:

```
# Búsqueda en el directorio src del frontend
grep -r "shakira\|Shakira\|lumineers\|bddshakira" frontend/src/ --include="*.jsx"
--include="*.js"
```

Resultado: 0 coincidencias. Las referencias al nombre del evento en el frontend provienen de los datos devueltos por la API (nombre del Punto de Venta, localidades), no de cadenas hardcodedas en el código fuente.

Se realizó también una búsqueda de URLs hardcodedas que pudieran apuntar a entornos incorrectos:

```
grep -r "localhost:5002\\|http://" frontend/src/ --include="*.jsx" --include="*.js"
```

Resultado: 0 coincidencias. Todas las llamadas a la API usan `import.meta.env.VITE_API_URL`, confirmando que el frontend no tiene URLs hardcodeadas de desarrollo.

8. Incidencias abiertas pendientes (fuera de este ciclo)

- **Credenciales en `render.yaml`:** se detectó desde la que este archivo de despliegue contiene una URI de MongoDB con usuario y contraseña en texto plano. No se ha corregido dentro de este ciclo porque afecta la configuración de despliegue en Render y requiere coordinación directa con el tutor empresarial para rotar credenciales y mover el valor a variables de entorno gestionadas desde el panel de Render.
- **Auditoría potencialmente duplicada:** detectada hoy en la revisión extendida (Sección 6.2). No bloquea el funcionamiento del sistema pero genera registros redundantes en la tabla de auditoría. Se documenta para evaluación en una futura iteración.

9. Conclusiones del día

Todas las incidencias funcionales y de seguridad detectadas durante el desarrollo (5 en total) quedan corregidas y verificadas, con casos de regresión manual documentados para cada una. La revisión de seguridad extendida sobre archivos no modificados previamente identificó cuatro hallazgos nuevos: auditoría potencialmente duplicada (`auditLogger.js`, sin impacto funcional inmediato), campo `usuario` buscado sin normalizar a minúsculas en `authController.js` (puede impedir el login con autocompletado del navegador), e interceptor de Axios en el frontend que enmascara el error de credenciales inválidas mostrando una pantalla en blanco en lugar del mensaje de error. Ninguno es bloqueante en el entorno local actual, pero los dos últimos tienen impacto real en producción y se recomiendan para corrección prioritaria antes del despliegue.